

LAB 4: Stochastic gradient descent - Documentation

The aim of this lab is to implement the design of Seminar 4, which extends our previous linear-regression framework with two new ideas: (1) **standardized datasets** using an abstract *Dataset* hierarchy, and (2) **stochastic gradient descent (SGD)** as an alternative optimization algorithm. We reused the classes *Vector*, *Record* and *Model* from previous labs, adapted *SupervisedLearner* to handle dataset transformations, and added new classes *Algorithm* (now abstract), *GradientDescent*, *StochasticGradientDescent*, *RawDataset* and *StandardizedDataset*. Finally, we created a *TestLearner* class to verify that all components worked correctly for both raw and standardized datasets.

Our program as a whole should be able to transform datasets (raw or standardized), compute either full-batch or stochastic gradients, learn a linear-regression model, and produce predictions on new inputs.

Classes, attributes, and methods

Classes we reused or slightly modified:

1. **Dataset** (abstract) → We made the class abstract and now it no longer computes means, standard deviations or z-scores. These are entirely delegated to *RawDataset*.
 - Attributes:
 - int dim
 - List<Record> data
 - Methods:
 - getDim(), getData()
 - Two abstract methods that all datasets must implement:
 - transform(Record r) → Record
 - output(double y) → double (reverse transformation)
2. **StandardizedDataset** (extends Dataset): Represents the dataset after z-score standardization.
 - Attributes:
 - Vector mi, si
 - double mo, so
 - Methods:
 - StandardizedDataset(d,mi,si,mo,so)
 - transform(Record r): Applies z-score transformation to input and output using ($x' = (x - mi) / si$) and ($y' = (y - mo) / so$)
 - output(double y): Applies the reverse transformation ($y = y' \cdot so + mo$)
3. **Algorithm** (abstract): Represents any supervised learning algorithm. (All learning algorithms must implement solve())
 - Attributes:
 - double learningRate
 - Methods:
 - Algorithm(double lr)
 - getLearningRate()
 - abstract solve(Dataset ds) → Model
4. **SupervisedLearner**: same as in lab 3 but with an important modification to the predict() method.
 - Changes:

- Before making a prediction, the input vector must be wrapped into a temporary Record so that `ds.transform()` can be applied.
- After getting the model's prediction (which may be in standardized space), we apply `ds.output()` to return a prediction in the original output space.
- This ensures the learner works both for raw and standardized datasets.

New classes we implemented:

5. **RawDataset** (extends Dataset): Represents an untransformed dataset and computes all statistics needed for standardization.
 - Attributes:
 - Vector `mi`, `si` (means and stds of inputs)
 - double `mo`, `so` (mean and std of outputs)
 - Methods:
 - Computes input/output means and stds
 - `transform(Record r)`: returns the identity (raw dataset does not modify records)
 - `output(double y)`: also identity
 - `standardize()`: computes all statistics and creates a StandardizedDataset
6. **GradientDescent** (extends Algorithm): Same implementation as the Algorithm class of Lab 3 but adapted to the new abstract design.
 - Attributes:
 - double `stoppingCriterion`
 - Methods:
 - `gradient(Dataset ds, Model m)` (full-batch gradient)
 - `solve(Dataset ds)` (iterative full gradient descent)
7. **StochasticGradientDescent** (extends Algorithm)
 - Attributes:
 - int `batchsize`
 - int `iterations`
 - Random `random`
 - Methods:
 - `stochasticGradient(Dataset ds, Model m)`: Samples a minibatch of records using `random.ints().distinct().limit()`, computes the average gradient.
 - `solve(Dataset ds)`: Runs SGD for a fixed number of iterations, updating the Model after each minibatch.
8. **TestLearner**: As in previous labs, we tested everything using “works / does not work” prints and an error counter.
 - Tests include:
 - Checking that raw datasets and standardized datasets transform correctly
 - Verifying stochastic gradients are not zero vectors
 - Testing that SGD and full gradient descent produce similar final models
 - Ensuring the reverse transformation (`output()`) works
 - Verifying predictions on simple linear datasets

Before coding, we discussed several possible designs. One alternative was to keep all mean/std computations inside Dataset like in Lab 2. This would have simplified RawDataset, but it would violate the abstraction introduced in Seminar 4, where Dataset is meant to act as a generic interface for transformed datasets. We preferred the required design because it cleanly separates responsibilities:

- RawDataset computes statistics and can generate StandardizedDataset
- Dataset defines the interface
- StandardizedDataset performs actual transformations

Another alternative was to compute minibatches by manually storing a list of sampled Records. Instead, we chose to use the Random.ints().distinct().limit() API as suggested in the handout. This avoids unnecessary data structures and makes the code more efficient and compact.

We also discussed whether to apply output() inside the Model class instead of SupervisedLearner. However, this would mix learning logic with dataset logic, breaking single responsibility and making Model depend on Dataset. We preferred the cleaner separation where SupervisedLearner handles all interactions between components.

This lab applied several object-oriented programming concepts:

- Inheritance: RawDataset and StandardizedDataset both extend Dataset and override its abstract methods.
- Abstraction: Dataset hides transformation logic behind abstract methods transform() and output().
- Delegation: SupervisedLearner delegates training to the Algorithm instance, which can be either full GD or SGD.
- Encapsulation: All attributes remain private, accessed only via getters.
- Polymorphism: Both GradientDescent and StochasticGradientDescent can be used interchangeably through the Algorithm interface.

In practice, our implementation worked well. TestLearner confirmed that:

- StandardizedDataset correctly produces mean ≈ 0 and std ≈ 1 after transformation.
- The modified predict() correctly handles transformed inputs and reverse-transforms outputs.
- GradientDescent converges to a stable model using the stopping criterion.
- StochasticGradientDescent, when properly tuned, produces a model similar to full gradient descent.
- Both raw and standardized datasets run without errors, and predictions stay consistent across both approaches.
- All classes integrated smoothly with those from previous labs.

We did face some difficulties during the implementation. One of the main challenges was adapting to the new Dataset hierarchy, since transforming the old Dataset into an abstract superclass required reorganizing logic and moving all statistical computations into RawDataset. Another issue was modifying the predict() method: forgetting to apply the reverse output transformation initially caused systematically wrong predictions on standardized datasets until we fixed it.

A particularly important part of the lab was tuning the hyperparameters for stochastic gradient descent. We experimented with several learning rates, and we chose the largest learning rate that

still produced a stable result, since too large values made the parameters diverge or oscillate. We also adjusted the number of iterations to the minimum value that still gave a stable output, to keep the algorithm efficient and to avoid unnecessary updates. This made SGD behave predictably while still being noticeably faster per iteration than full gradient descent.

After fixing these issues and selecting good hyperparameters, the final solution passed all our tests with 0 errors. Overall, this lab helped us better understand abstraction, inheritance, and how stochastic optimization fits into an object-oriented framework, as well as the practical side of tuning learning algorithms.